A Racing Game with Machine Learning Bots as Opponents

Project Plan

Karan Dhillon

CS3810

Supervised By: Dr Adrian Johnstone
Department of Computer Science
Royal Holloway, University of London

1 Abstract

My project objective is to create a racing game using ML agents as opponents. I intend to train ML agents using the unity ML agents toolkit[4] to provide an engaging and challenging racing experience for the user. I intend to model a more advanced suspension model as opposed to the built in WheelCollider to provide a more enjoyable and more reliable suspension to maximise user enjoyment while also providing a demonstration of my coding ability as a software engineering student. One potential option is to implement multiple raycasts to avoid the problems associated with a single raycast suspension model found in wheel colliders[1]. I would implement this by using the raycasts to find how close objects are from the centre of the wheel using the Physics.Raycast[3] and its distance attribute, to avoid clipping issues of the wheel collider physics object. Using readily available formulae and diagrams[6], I can use these raycasts to harvest collision data and use it to accurately simulate the forces applied on the car's bodyshell, the forces on the ground from the wheels.

This game will feature races that will take the form of a simple point to point event with checkpoints to ensure both the user and the ML drivers follow the route and to allow the user to reset to the last checkpoint if they get stuck or drive off course. Additionally, I will add a simpler implementation of a race with no ML opponents, where the user races against the clock. This can be used as a testing ground for the checkpoint and timer system, while also being able to be used as a game mode by the end user. The game will also have a cash system to use as a reward for participating in, and winning, races and time trials. This cash system would be used to buy new cars to enter vehicle specific events.

I chose Unity because Unity is much more beginner friendly than unreal, and it has a large number of community made tutorials and forum replies. I initially wanted to use godot, however upon further research I learned that Godot's 3D abilities are not as advanced as unity, which is why I elected to use unity over Godot. There are also many racing games made by amateur developers made using Unity[2], which is why I chose it as it would be a good base for my own racing game. One limitation I found while researching this engine for my project is the limitations of the inbuilt physics objects for cars, specifically with the WheelCollider class. With this limitation in mind, I intend to bypass this class and model my own suspension for the cars in my racing game. Another benefit of using the unity engine is being able to take advantage of its integration with Nvidia PhysX, which includes vehicle representation structures that can help speed up development and testing[5].

2 TIMELINE

2.1 Term 1

Week 1 - 2: Research and Complete suspension model for car
Week 3 - 4: Create a car controller to allow for the car to be controlled via the user or methods that can be accessed by an ML agent
Week 5: Design a Basic Track
Week 6: implement the Time Trial mode
Week 7: Implement a Cash system and Switchable Current Cars
Week 8: Implement a dealership system to test switching, buying and selling cars
Week 9: Implement a save system
Week 10 - 11: Research and attempt to implement ML Drivers in preparation of implementing a racing system

3 Risks and Mitigations

3.1 Protection against poorly written or buggy code

By implementing Unit Testing and TDD to ensure that my code is robust and consistently tested. Furthermore, by implementing a CI/CD Pipeline, I can make sure every commit is tested to ensure I have not inadvertently broken a piece of code in a different part of my project.

3.2 Poor code performance

By consistently playtesting my code, I can ensure the code will run to a satisfactory speed. This will prevent features being finished without being thoroughly tested for efficiency.

3.3 Poor estimation of tasks

By prioritising the research of how to complete a task before attempting it, I will know if I will need to spend extra time on a task or exchange time for a lesser task for taxing tasks

3.4 Hardware failure

By keeping constant backups of my game assets, and using the RHUL CIM Gitlab to version control my code, I can keep my code independent of any single machine, which can insulate my project from the consequences of hardware failure.

3.5 Unreasonable Scope

By focusing my attention to the core features of my project, I will be able to reduce the chance of unreasonable scope to slow the progress of my project. As there are few core features to focus on, by having the basis of the game ready for the ML agents to be trained on as soon as possible, I can give myself as much time as possible to complete my project.

3.6 Ensuring Testing Features does not affect my project

By developing complex features, (such as vehicle suspension) into a separate Unity Project. This will allow me to focus more on the feature being developed instead of trying to work around the code and assets already implemented in my project

Acronyms

ML - Machine Learning
TDD - Test Driven Development
CI/CD Continuous Integration / Continuous Deployment
RHUL - Royal Holloway University of London
CIM - Computer Science, Information Security, Mathematics

Glossary

Unity - Game Engine I will be using for this project
Gitlab - Version Control provided by RHUL CIM, based of Github
Raycast - A “ray” sent from a given position vector for the purpose of collision detection

References

[1] - Unity Documentation. (n.d.). *‘Prepare the ground for raycast Wheel Colliders’*. [online] Available at: https://docs.unity3d.com/2022.3/Documentation/Manual/wheel-colliders-introduction.html.
[2] - Singh, S. and Kaur, A. (2022). *Game Development using Unity Game Engine*. [online] IEEE Xplore. doi:https://doi.org/10.1109/ICAN56228.2022.10007155 Section 6 - A.
[3] - Unity Documentation. (n.d.). *Physics.Raycast*. [online] Available at: https://docs.unity3d.com/ScriptReference/Physics.Raycast.html.
[4] - GitHub. (n.d.). Available at: https://github.com/Unity-Technologies/ml-agents.
[5] - Templet, R. (2020). *Game Physics: An Analysis of Physics Engines for First-Time Physics Developers*. [online] p.55. Available at: https://scholarworks.calstate.edu/downloads/z890s004h.
[6] - Pratheepa, B. (2010). Modeling and simulation of automobile suspension system. *Frontiers in Automobile and Mechanical Engineering -2010*, 2A. doi:https://doi.org/10.1109/fame.2010.5714866.